Preprocessing before building machine learning models

Lino Galiana
Version of October 20, 2026

In order to obtain data that is consistent with modeling assumptions, it is essential to take the time to prepare the data to be supplied to a model. The quality of the prediction depends heavily on this preliminary work, known as *preprocessing*. This chapter presents the issues involved and illustrates them using the `Scikit Learn` library, which makes this work less tedious and more reliable.

⚠️ **Automatically generated PDF.** This document is produced from the course website and may contain formatting issues. For an up-to-date version adapted to your reading, visit the course page: pythonds.linogaliana.fr.

Table of contents

1 Introduction **2**
1.1 Introduction to the `Scikit` ecosystem 2
1.2 Data preparation 2
2 General Approach **6**
3 Exploring data structure **7**
4 Transforming data **11**
4.1 Continuous Variable Preprocessing 12
4.2 Encoding Categorical Values 19
4.3 Imputation 19
4.4 Handling Outliers 19
4.5 Application exercise 20
Reference **21**
Informations additionnelles 21
Bibliography **23**

Skills you will acquire in this chapter

- Understand the critical role of preprocessing in aligning data with modeling assumptions, using the robust Scikit-Learn ecosystem
- Use Pandas to explore data structure and select relevant features before modeling
- Transform continuous variables to fit model requirements: standardization (zero mean, unit variance) or normalization (unit norm), depending on the algorithm
- Encode categorical variables using `LabelEncoder`, `OrdinalEncoder`, or `OneHotEncoder` to make them usable in machine learning models
- Handle missing data through imputation (mean, median, mode, or more advanced techniques) rather than simply dropping observations
- Detect and manage outliers by analyzing their distribution and removing them when they negatively impact model performance
- Understand that in Scikit-Learn, preprocessing is part of the learning process: parameters like mean or variance are estimated from the training set and reapplied to new data, ensuring consistency in distribution between training and prediction phases

1 Introduction

The introduction to this section discussed the importance of adopting an algorithmic rather than a statistical approach for modeling empirical processes. The goal of this chapter is to introduce machine learning methodology and the choices that an algorithmic approach entails for structuring data work. This will also be an opportunity to introduce the Python machine learning ecosystem, particularly its core library: `Scikit-Learn`.

The aim of this chapter is to present some data preparation elements. This is a fundamental step that should not be overlooked. Models are based on certain assumptions, usually related to the theoretical distribution of variables, which are integrated into them.

It is necessary to align the empirical distribution with these assumptions, which requires a restructuring of the data. This will lead to more relevant modeling results. In the chapter on *pipelines*, we will see how to industrialize these preprocessing steps to simplify applying a model to a dataset different from the one on which it was estimated.

This chapter, like the entire machine learning section, is a practical introduction illustrated from an electoral prediction perspective. Specifically, it involves predicting the results of the 2020 U.S. elections at the county level based on socio-demographic variables. The underlying idea is that there are sociological, economic, or demographic factors influencing voting behavior, but the motivations or complex interactions between these factors are not well understood.

1.1 Introduction to the `Scikit` ecosystem

`Scikit Learn` is currently the go-to library in the machine learning ecosystem. It is a library that, despite its many implemented methods, offers the advantage of a unified entry point. This unified approach is one of the reasons for its early success. `R` only recently gained a unified library, namely `tidymodels`.

Another reason for `Scikit`'s success is its operational focus: deploying models developed through `Scikit` pipelines is cost-effective. A dedicated chapter of this course covers pipelines. Together with Romain Avouac, we offer a more advanced course in the final year at ENSAE, where we present some challenges related to deploying models developed with `Scikit`.

The `Scikit` user guide is a valuable reference to consult regularly. The section on *preprocessing*, the focus of this chapter, is available here.

> ℹ Scikit Learn, a French Success! 🐓🥖🥐
>
> `Scikit Learn` is an open-source library originating from the work of Inria 🇫🇷. For over 10 years, this French public institution has developed and maintained this package, which is downloaded 2 million times a day. In 2023, to secure the maintenance of this package, a startup named `Probabl.ai` was created around the team of `Scikit` developers.
>
> To explore the depth of the `Scikit` ecosystem, it is recommended to follow the `Scikit MOOC`, developed as part of the `Inria Academy` initiative.

1.2 Data preparation

Exercise 1 allows those interested to try to recreate it step by step.

The following packages are needed to import and visualize the election data:

```
!pip install geopandas
```

The data sources are varied, so the code that builds the final dataset is provided directly. Building a single dataset can be somewhat tedious, but it's a good exercise, which you can try, to review `Pandas`:

> Exercise 1 (Optional): Build the Database
>
> **This exercise is OPTIONAL**
>
> 1. Download and import the shapefile from this link
> 2. Exclude the following states: "02", "69", "66", "78", "60", "72", "15"
> 3. Import election results from this link
> 4. Import the datasets available on the USDA site, ensuring that FIPS code variables are named consistently across all 4 datasets
> 5. *Merge* these 4 datasets into a single socioeconomic characteristics dataset
> 6. *Merge* with the election data using the FIPS code
> 7. *Merge* with the shapefile using the FIPS code. Be mindful of leading zeros in some codes. It is recommended to use the `str.lstrip` method to remove them
> 8. Import election data from 2000 to 2016 from the MIT Election Lab. The data can be directly queried from this URL: https://minio.lab.sspcloud.fr/lgaliana/data/python-ENSAE/countypres_2000-2024.csv (manual export from Harvard dataverse)
> 9. Create a `share` variable to account for each candidate's vote share. Keep only the columns `"year", "FIPS", "party", "candidatevotes", "share"`
> 10. Perform a `long` to `wide` conversion using the `pivot_table` method to keep one row per county x year with each candidate's results in columns for that state.
> 11. Merge with the rest of the dataset using the FIPS code.

```
import requests

url     =     'https://raw.githubusercontent.com/linogaliana/python-datascientist/main/content/
modelisation/get_data.py'
r = requests.get(url, allow_redirects=True)
open('getdata.py', 'wb').write(r.content)

import getdata
votes = getdata.create_votes_dataframes()
```

However, before focusing on data preparation, we will spend some time exploring the structure of the data from which we want to build a model. This is essential to understand its nature and choose an appropriate model.

This code introduces a dataset named `votes` into the environment. It is a combined dataset from different sources and appears as follows:

```
votes.loc[:, votes.columns != "geometry"].head(3)
```

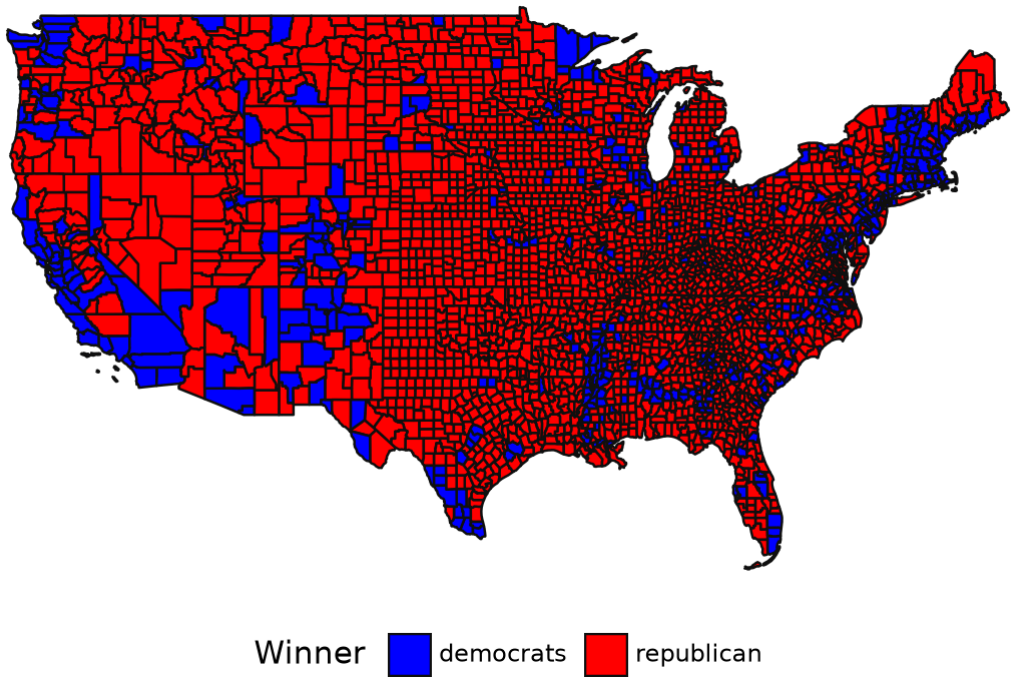
	STATEFP	COUNTYFP	COUNTYNS	AFFGEOID	GEOID	NAME	LSAD	ALAND	AWATER	FIPS_x	...	share_2020_GREEN	share_2020_LIBERTARIAN	share_2020_OTHER	share_2020_REPUBLICAN	share_2024_DEMOCRAT	share_2024_LIBERTARIAN	share_2024_OTHER	share_2024_REPUBLICAN	share_2024_other	winner
0	29	227	00758566	0500000US29227	29227	Worth	06	690564983	493903	29227	...	0.000000	0.012647	0.000903	0.792231	0.171946	0.008145	0.001810	0.818100	NaN	republican
1	31	061	00835852	0500000US31061	31061	Franklin	06	1491355860	487899	31061	...	NaN	0.006381	NaN	0.833527	0.147500	0.005625	0.002500	0.844375	NaN	republican
2	36	013	00974105	0500000US36013	36013	Chautauqua	06	2746047476	1139407865	36013	...	0.003396	0.012648	0.014573	0.583119	0.389156	0.000775	0.001656	0.608412	NaN	republican

The following choropleth map provides a quick visualization of the results (Alaska and Hawaii are excluded).

```
from plotnine import *
```

```
# republican : red, democrat : blue
color_dict = {'republican': '#FF0000', 'democrats': '#0000FF'}

(
  ggplot(votes) +
  geom_map(aes(fill = "winner")) +
  scale_fill_manual(color_dict) +
  labs(fill = "Winner") +
  theme_void() +
  theme(legend_position = "bottom")
)
```



! The Territorial Trap

As mentioned in the chapter on mapping, choropleth maps can give a misleading impression that the Republican Party won by a large margin in 2020 because this type of graphic representation gives more importance to large areas rather than dense areas. This explains why this type of map has been used as justification for contesting the election results.

There are alternative representations better suited to such phenomena where density is significant. One such representation is proportional circles (see Insee [1], *"The Territorial Trap in Cartography"*). Proportional circles allow the eye to focus on denser areas rather than large open spaces. With this representation, it becomes clear that the Democratic vote is in the majority, which was obscured by the flat color fill.

The GIF "Land does not vote, people do", which gained popularity in 2020, offers another visualization approach. The original map was created with `JavaScript`. However, with `Python`, we have several tools to replicate this map at a low cost using one of the `JavaScript` overlays discussed in the visualization section.

```
import numpy as np
import pandas as pd
import geopandas as gpd
```

```
import plotly
import plotly.graph_objects as go


centroids = votes.copy()
centroids.geometry = centroids.centroid
centroids['size'] = centroids['CENSUS_2020_POP'] / 10000  # to get reasonable plotable number

color_dict = {"republican": '#FF0000', 'democrats': '#0000FF'}
centroids["winner"] =  np.where(centroids['votes_gop'] > centroids['votes_dem'], 'republican',
'democrats')


centroids['lon'] = centroids['geometry'].x
centroids['lat'] = centroids['geometry'].y
centroids = pd.DataFrame(
  centroids.loc[
    : ,
    ["county_name",'lon','lat','winner', 'CENSUS_2020_POP',"state_name"]
  ]
)
groups = centroids.groupby('winner')

df = centroids.copy()

df['color'] = df['winner'].replace(color_dict)
df['size'] = df['CENSUS_2020_POP']/6000
df['text']   =   df['CENSUS_2020_POP'].astype(int).apply(lambda   x:   '<br>Population:   {:,}
people'.format(x))
df['hover']  =  df['county_name'].astype(str)  +    df['state_name'].apply(lambda  x:  '  ({})
'.format(x)) + df['text']

fig_plotly = go.Figure(
  data=go.Scattergeo(
  locationmode = 'USA-states',
  lon=df["lon"], lat=df["lat"],
  text = df["hover"],
  mode = 'markers',
  marker_color = df["color"],
  marker_size = df['size'],
  hoverinfo="text"
  )
)

fig_plotly.update_traces(
  marker = {'opacity': 0.5, 'line_color': 'rgb(40,40,40)', 'line_width': 0.5, 'sizemode': 'area'}
)

fig_plotly.update_layout(
   title_text = "Reproduction of the \"Acres don't vote, people do\" map <br>(Click legend to
toggle traces)",
  showlegend = True,
  geo = {"scope": 'usa', "landcolor": 'rgb(217, 217, 217)'}
)

fig_plotly.show()
```

```
/tmp/ipykernel_335/1328083585.py:9: UserWarning: Geometry is in a geographic CRS. Results from
'centroid' are likely incorrect. Use 'GeoSeries.to_crs()' to re-project geometries to a projected
CRS before this operation.
```

```
Unable to display output for mime type(s): text/html
```

```
Unable to display output for mime type(s): text/html
```

2 General Approach

In this chapter, we will focus on data preparation to be done before modeling work. This step is essential to ensure consistency between the data and modeling assumptions and to produce scientifically valid analyses.

The general approach we will adopt in this chapter, which will be refined in subsequent chapters, is as follows:

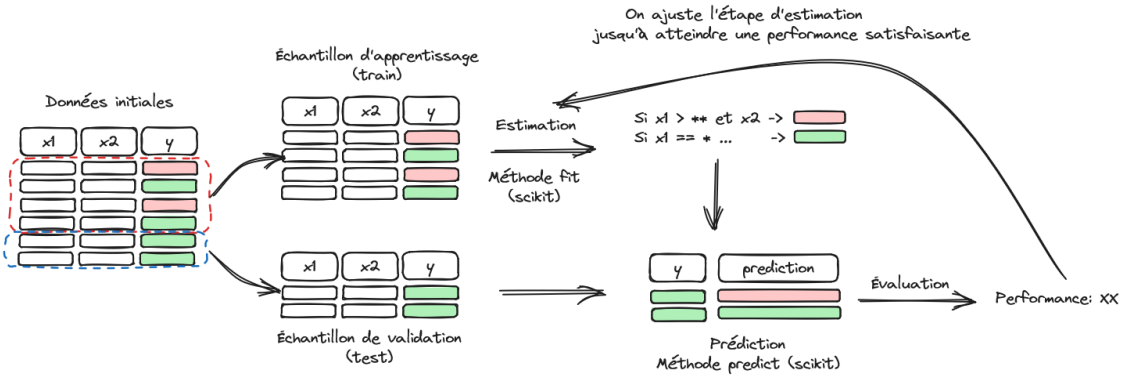


Figure 1: Illustration of machine learning methodology

Figure 1 illustrates the structure of a machine learning problem.

First, the available dataset is split into two parts: **training sample** and **validation sample**. The former is used to train a model, and its prediction quality is evaluated on the latter to limit overfitting bias. The next chapter will delve deeper into model evaluation. At this stage of our progress, we will focus in this chapter on data issues.

The `Scikit` library is particularly convenient because it offers many machine learning algorithms with a few unified entry points, especially the `fit` and `predict` methods. However, the unification extends beyond algorithm training. All data preparation steps integrated into `Scikit` offer these same entry points. In other words, data preparations are built like parameter estimations that can be reapplied to another dataset. For example, this data preparation might be a mean and variance estimation for normalizing variables. The mean and variance are calculated on the training sample, and the same values can be reapplied to another dataset to normalize it in the same way.

3 Exploring data structure

The first necessary step before diving into modeling is to determine which variables to include in the model.

`Pandas` functionalities are sufficient at this stage for exploring simple structures. However, when dealing with a dataset with numerous explanatory variables (*features* in machine learning, *covariates* in econometrics), it is often wise to start with a variable selection step, which we will cover later in the dedicated section.

Before selecting the best set of explanatory variables, we will start with a small and arbitrary selection. The first task is to represent the relationships within the data, particularly the relationship between explanatory variables and the dependent variable (the Republican Party's score), as well as relationships among explanatory variables.

For the next exercise, to illustrate the principle of visual inspection of correlations, we'll keep only a limited number of variables, chosen somewhat arbitrarily.

```
df2 = votes.set_index("GEOID").loc[
    : ,
    [
        "winner", "votes_gop",
        'Unemployment_rate_2019', 'Median_Household_Income_2021',
        'Percent of adults with less than a high school diploma, 2018-22',
        "Percent of adults with a bachelor's degree or higher, 2018-22"
    ]
]
df2 = df2.dropna()
```

Exercise 2 (Optional): Examining Correlations Between Variables

This exercise is OPTIONAL

Use a graph to represent the correlation matrix. You can use the `seaborn` package and its `heatmap` function.

The correlation matrix can be constructed in various ways, depending on the framework you choose (see the chapter on visualization). The example below was produced using `seaborn`, which is well-suited for quick visualizations. However, considerable refinement would be needed to make the graphic suitable for publication.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

column_labels = {
    'votes_gop': 'GOP votes',
    'Unemployment_rate_2019': 'Unemployment\n(2019)',
    'Median_Household_Income_2021': 'Median Income\n(2021)',
   'Percent of adults with less than a high school diploma, 2018-22': 'Less than HS diploma\n(2018-
22)',
     'Percent of adults with a bachelor\'s degree or higher, 2018-22': 'Bachelor\'s degree
or\nhigher (2018-22)'
}

corr = (
    df2.drop("winner", axis = 1)
```

```
    .rename(columns=column_labels)
    .corr()
)

mask = np.zeros_like(corr, dtype=bool)
mask[np.triu_indices_from(mask)] = True

# Set up the matplotlib figure
fig = plt.figure()

# Generate a custom diverging colormap
cmap = sns.diverging_palette(220, 10, as_cmap=True)

# Draw the heatmap with the mask and correct aspect ratio
g = sns.heatmap(
    corr,
    mask=mask,     # Mask upper triangular matrix
    cmap=cmap,
    annot=True,
    vmax=.3,
    vmin=-.3,
    center=0,      # The center value of the legend. With divergent cmap, where white is
    square=True,
    linewidths=.5,
    cbar_kws={"shrink": .5}
)

g
plt.xticks(rotation=30, ha='right')
plt.yticks(rotation=0)
plt.show()
```

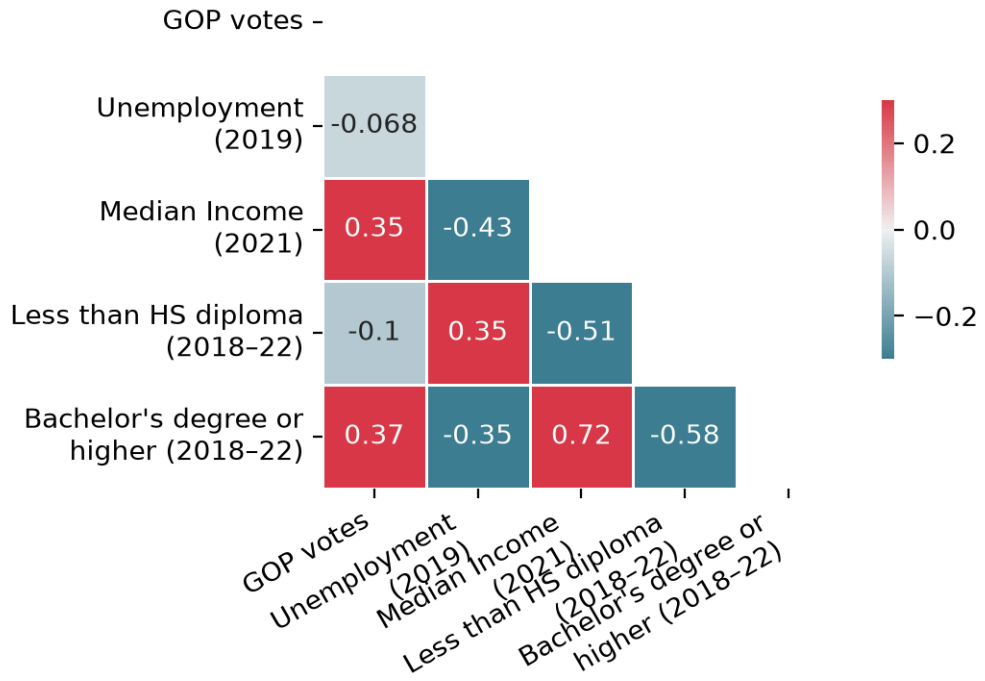


Figure 2: Correlation matrix with seaborn

A similar but slightly cleaner figure can be produced using `plotnine`, the `Python` equivalent of the `ggplot2` library in `R` (for more details, see the chapter on visualization). Since there is less online inspiration for `plotnine` - which is newer than `matplotlib` or `seaborn` - and fewer examples available compared to the rich ecosystem around `R`, the full code is provided below[1].

```
corr = (
    df2.drop("winner", axis = 1)
    .rename(columns=column_labels)
    .corr()
)

# 2. Transformation en format long
corr_long = corr.reset_index().melt(id_vars='index')
corr_long.columns = ['var1', 'var2', 'corr']

# 3. Supprimer la diagonale supérieure
corr_long['mask'] = corr_long.apply(lambda row: sorted([row['var1'], row['var2']]), axis=1)
corr_long['keep'] = corr_long['mask'].duplicated(keep='first')
corr_long = corr_long[~corr_long['keep']].drop(columns=['mask', 'keep'])

# 4. Ordre cohérent
var_order = corr.columns.tolist()
corr_long['var1'] = pd.Categorical(corr_long['var1'], categories=var_order, ordered=True)
corr_long['var2'] = pd.Categorical(corr_long['var2'], categories=var_order[::-1], ordered=True)

# 5. Groupe pour texte contrasté
corr_long['p_group'] = np.where(corr_long['corr'].abs() > 0.2, 'white', 'black')

# 5. Création du plot
p = (
    ggplot(corr_long, aes(x='var1', y='var2', fill='corr'))
    + geom_tile(width=0.95, height=0.95)
    + geom_text(aes(label='round(corr, 2)', color='p_group'), size=9, show_legend=False)
    + scale_fill_gradient2(low='blue', mid='white', high='red', midpoint=0, limits=(-1, 1))
    + scale_color_manual(values={'white': 'white', 'black': 'black'})
    + coord_fixed()
    + labs(x = "", y = "")
    + theme(
        axis_text_x=element_text(rotation=45, ha='right'),
        figure_size=(8, 6),
        panel_background=element_rect(fill='white'),
        legend_position = "none"
    )
)

p
```

[1] This version was generated with the help of `ChatGPT` in just a few seconds, sparing me the trouble of figuring out the ideal data structure to replicate the *heatmap* example from the `plotnine` documentation.

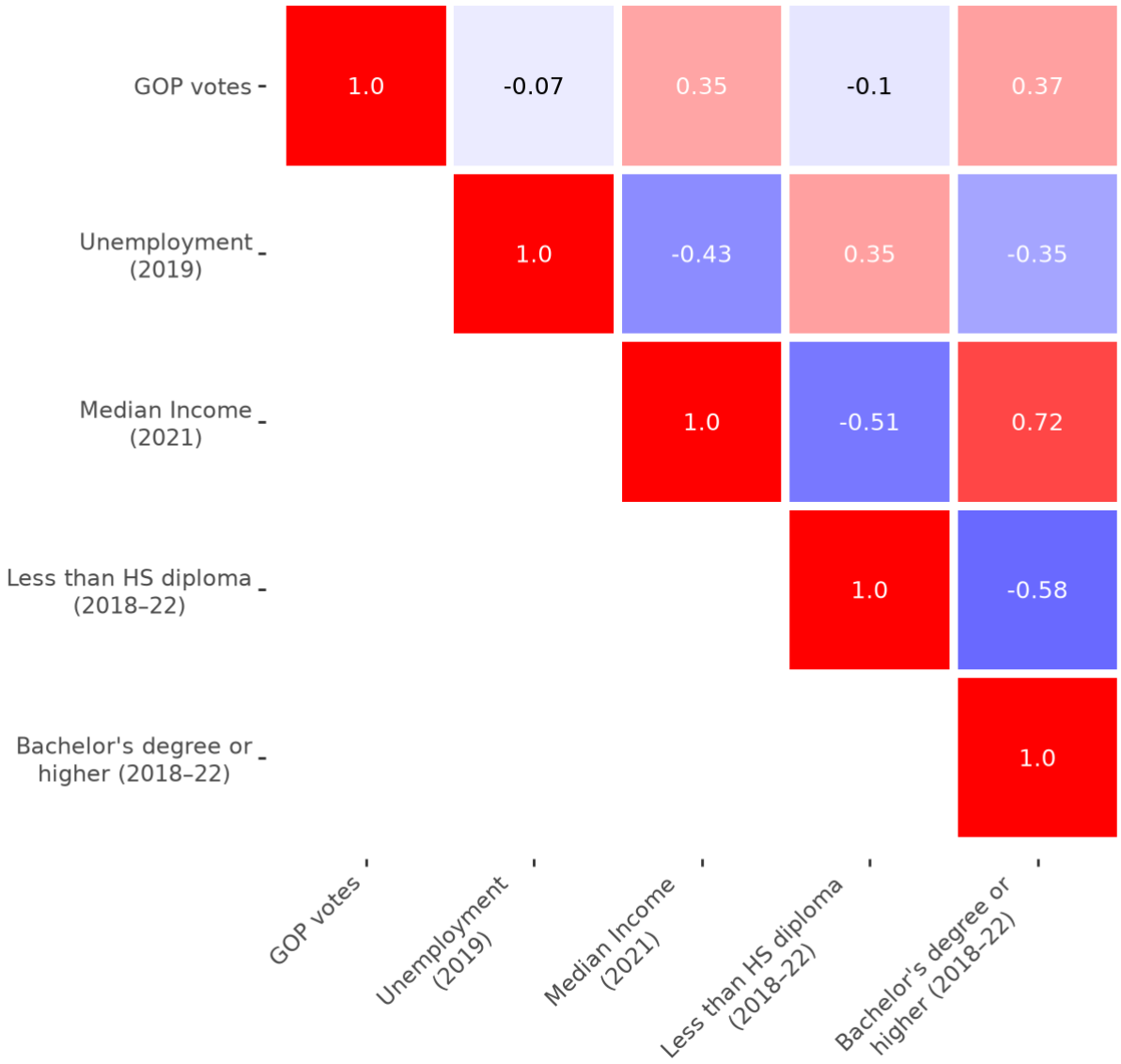


Figure 3: Correlation matrix with plotnine

Last but not least, you can also build this correlation matrix with `Plotly`, even if it's not the most practical ecosystem.

```
import plotly.express as px
import pandas as pd
import numpy as np

# Compute correlation matrix
corr = (
    df2.drop("winner", axis=1)
    .round(2)
    .rename(columns=column_labels)
    .corr()
)

# Mask upper triangle
mask = np.triu(np.ones_like(corr, dtype=bool))
corr_masked = corr.mask(mask)
```

```
# Plot heatmap
fig = px.imshow(
    corr_masked.values,
    x=corr.columns,
    y=corr.columns,
    color_continuous_scale='RdBu_r',  # ← reversed color scale
    zmin=-1,
    zmax=1,
    text_auto=".2f"
)

# Customize hover
fig.update_traces(
    hovertemplate="Var 1: %{y}<br>Var 2: %{x}<br>Corr: %{z:.2f}<extra></extra>"
)

# Layout tweaks
fig.update_layout(
    coloraxis_showscale=False,
    xaxis=dict(showticklabels=False, title=None, ticks=''),  # remove axis title and ticks
    yaxis=dict(showticklabels=True, title=None, ticks=''),
    plot_bgcolor="rgba(0,0,0,0)",
    margin=dict(t=10, b=10, l=10, r=10),  # ⟵ shrink ALL margins
    width=600,
    height=600
)

fig.show()
```

```
Unable to display output for mime type(s): text/html
```

Figure 4: Correlation matrix built with plotly

As expected, the strongest correlations (in absolute value) are between income and education level. Income is also positively correlated with the Republican vote share. However, these results are purely descriptive and do not account for potential confounding factors. To explore these relationships more rigorously, we'll need to introduce a modeling approach that can control for the interplay between variables.

4 Transforming data

Differences in scale or distribution between variables can diverge from the underlying assumptions in models.

For example, in linear regression, categorical variables are not treated the same way as variables with values in $\mathbb{R}$. A discrete variable (taking a finite number of values) must be transformed into a sequence of 0/1 variables (dummies) relative to a reference category to meet the assumptions of linear regression. This type of transformation is known as *one-hot encoding*, which we will revisit. It is one of many transformations available in `Scikit` to align a dataset with mathematical assumptions.

All these data preparation tasks fall under preprocessing or feature engineering. One advantage of using `Scikit` is that preprocessing tasks can be considered learning tasks. Preprocessing involves learning parameters from a data structure (e.g., estimating means and variances to subtract from

each observation), and these parameters can then be applied to observations not used to calculate them. In other words, this data preparation fits seamlessly into the pipeline shown in Figure 1.

4.1 Continuous Variable Preprocessing

We will cover two very common preprocessing steps for continuous variables:

1. **Standardization** transforms data so that the empirical distribution follows a $\mathcal{N}(0,1)$ distribution.
2. **Normalization** transforms data to achieve a unit norm (ℓ_1 or ℓ_2). In other words, with the appropriate norm, the sum of elements equals 1.

There are other options, such as `MinMaxScaler` to rescale variables according to observed minimum and maximum bounds. The choice of method depends on the algorithms chosen later: the assumptions of k-nearest neighbors (knn) differ from those of a random forest. For this reason, complete pipelines are usually defined, integrating both preprocessing and learning, which will be discussed in upcoming chapters.

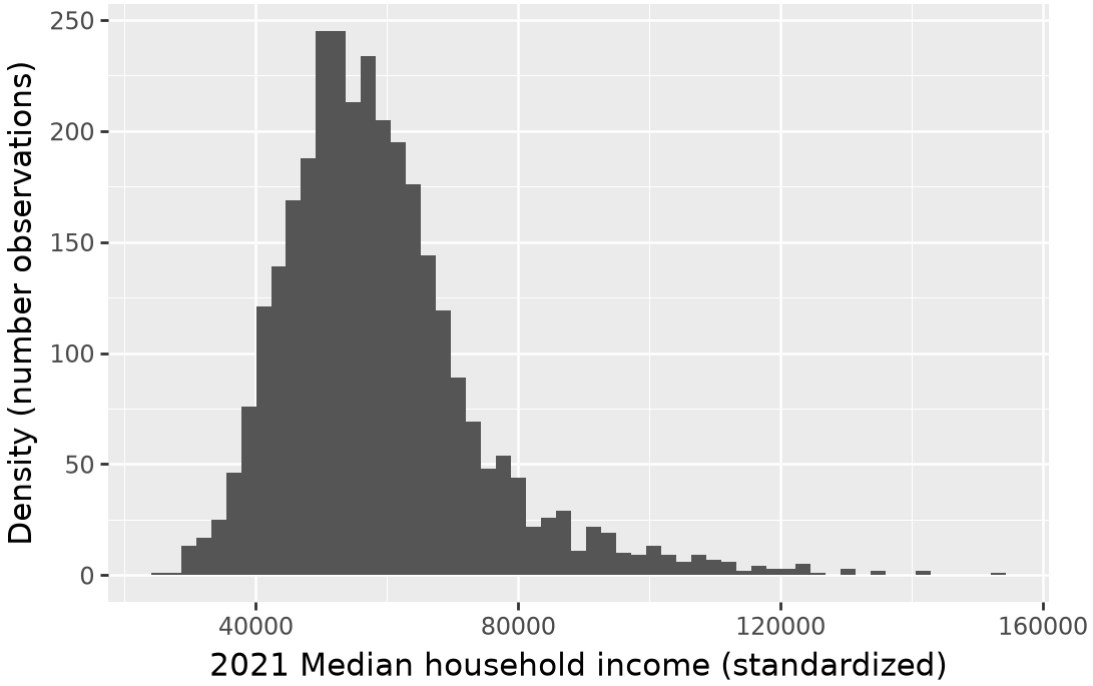
> Caution
>
> For statisticians, the term *normalization* in `Scikit` terminology can have a counterintuitive meaning. One might expect normalization to transform a variable so that $X \sim \mathcal{N}(0,1)$. In `Scikit`, this is actually **standardization**.

4.1.a Standardization

Standardization involves transforming data so that the empirical distribution follows a $\mathcal{N}(0,1)$ distribution. For optimal performance, most machine learning models often require data to follow this distribution. Even when it's not essential, as with logistic regressions, it can speed up the convergence rate of algorithms.

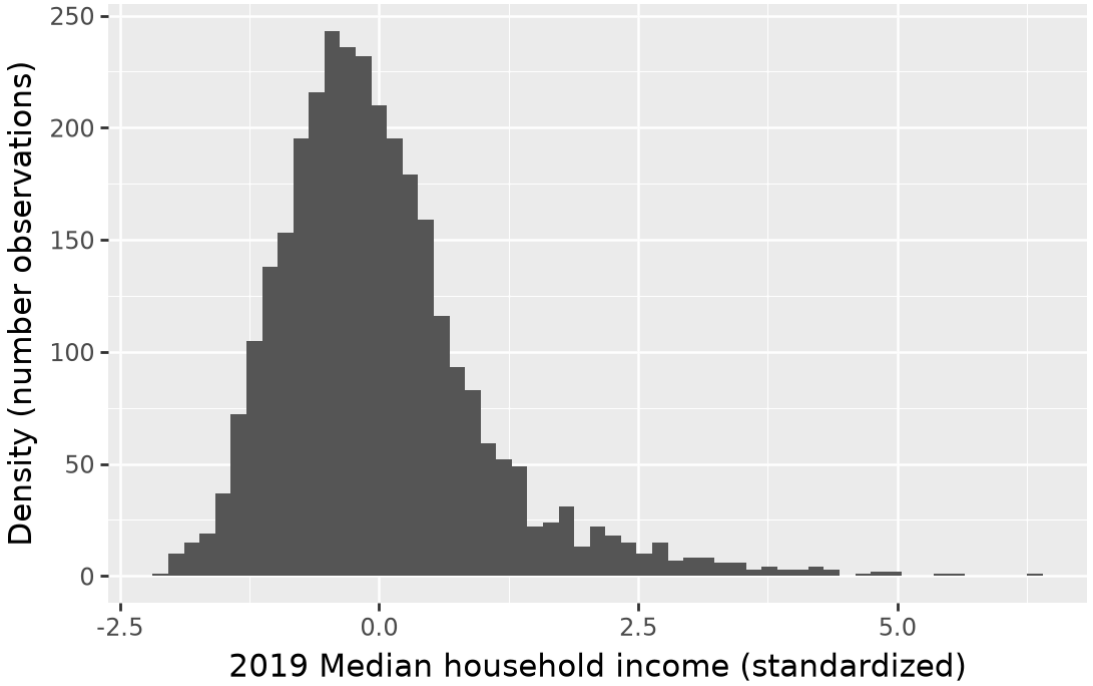
> Exercise 3: Standardization
>
> 1. Standardize the `Median_Household_Income_2021` variable (do not overwrite the values!) and examine the histogram before and after normalization. This transformation should be applied to the entire column; the next questions will address sample splitting and extrapolation.
>
> *Note: This should yield a distribution centered at zero, and we could verify that the empirical variance is indeed 1. We could also check that this is true when transforming multiple columns at once.*
>
> 2. Create `scaler`, a `Transformer` built on the first 1000 rows of your `df2` DataFrame, excluding the target variable `winner`. Check the mean and standard deviation of each column for these same observations.
>
> *Note: The parameters used for subsequent standardization are stored in the `.mean_` and `.scale_` attributes.*
>
> These attributes can be seen as parameters trained on a specific dataset that can be reused on another, as long as the dimensions match.
>
> 3. Apply `scaler` to the remaining rows of the DataFrame and compare the distributions obtained for the `Median_Household_Income_2021` variable.
>
> *Note: Once applied to another `DataFrame`, you may notice that the distribution is not exactly standardized in the DataFrame on which the parameters were not estimated. This is normal; the initial sample was not random, so the means and variances of this sample do not necessarily match those of the complete sample.*

Before standardization, our variable has this distribution:



After standardization, the scale of the variable has changed.

```
/__w/python-datascientist/python-datascientist/.venv/lib/python3.13/site-packages/plotnine/
stats/stat_bin.py:112: PlotnineWarning: 'stat_bin()' using 'bins = 57'. Pick better value with
'binwidth'.
```



We indeed obtain a mean equal to 0 and a variance equal to 1, within numerical approximations:

	Statistique	Valeur
0	Mean	0.000000
1	Variance	1.000323

For question 2, if we attempt to present the obtained statistics in a readable table, we get

Variable	Mean before Scaling	Std before Scaling	Mean after Scaling	Std after Scaling
votes_gop	17.4K	32.8K	−3.4E−17	1.00
Unemployment_rate_2019	3.81	1.29	1.1E−16	1.00
Median_Household_Income_2021	52.3K	14.0K	1.2E−16	1.00
Percent of adults with less than a high school diploma, 2018-22	11.7	5.97	1.6E−16	1.00
Percent of adults with a bachelor's degree or higher, 2018-22	23.0	9.79	2.6E−16	1.00
y_standard	−0.026	0.92	3.6E−18	1.00

It is very clear from this table that the standardization worked well.

Now, if we create a formal transformer for our variables (question 3)

	copy **copy: bool, default=True** **If False, try to avoid a copy and do inplace scaling instead.** **This is not guaranteed to always work inplace; e.g. if the data is** **not a NumPy array or scipy.sparse CSR matrix, a copy may still be** **returned.**	**True**
	with_mean with_mean: bool, default=True If True, center the data before scaling. This does not work (and will raise an exception) when attempted on sparse matrices, because centering them entails building a dense matrix which in common use cases is likely to be too large to fit in memory.	True
	with_std with_std: bool, default=True	True

	If True, scale the data to unit variance (or equivalently, unit standard deviation).	

Name	Type	Value
feature_names_in_ feature_names_in_: ndarray of shape (`n_features_in_`,) Names of features seen during :term:`fit`. Defined only when `X` has feature names that are all strings. .. versionadded:: 1.0	ndarray[object](6,)	['votes_gop','Unemployment_rate_2019','Median_Housel 'Percent of adults with less than a high school diploma, 2018-22', "Percent of adults with a bachelor's degree or higher, 2018-22", 'y_standard']
mean_ mean_: ndarray of shape (n_features,) or None The mean value for each feature in the training set. Equal to ``None`` when ``with_mean=False`` and ``with_std=False``.	ndarray[float64](6,)	[17361.65, 3.81,58348.15, 11.67, 22.98, −0.03]
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	6
n_samples_seen_ n_samples_seen_: int or ndarray of shape (n_features,) The number of samples processed by the estimator for each feature. If there are no missing samples, the ``n_samples_seen`` will be an integer, otherwise it will be an array of dtype int. If `sample_weights` are used it will be a float (if no missing data) or an array of dtype float that sums the weights seen so far. Will be reset on new calls to fit, but increments across ``partial_fit`` calls.	float64	1000
scale_ scale_: ndarray of shape (n_features,) or None	ndarray[float64](6,)	[32794.96, 1.29,14018.77, 5.97, 9.79, 0.92]

Per feature relative scaling of the data to achieve zero mean and unit variance. Generally this is calculated using `np.sqrt(var_)`. If a variance is zero, we can't achieve unit variance, and the data is left as-is, giving a scaling factor of 1. `scale_` is equal to `None` when `with_std=False`. .. versionadded:: 0.17 *scale_*		
var_ var_: ndarray of shape (n_features,) or None The variance for each feature in the training set. Used to compute `scale_`. Equal to ``None`` when ``with_mean=False`` and ``with_std=False``.	ndarray[float64](6,)	[1.08e+09,1.67e+00,1.97e+08,3.57e+01,9.58e+01,8.52e-01]

votes_gop
Unemployment_rate_2019
Median_Household_Income_2021
Percent of adults with less than a high school diploma, 2018-22
Percent of adults with a bachelor's degree or higher, 2018-22
y_standard

We can extrapolate our standardizer to a larger dataset. If we look at the distribution obtained on the first 1000 rows (question 3), we find a scale consistent with a $\mathcal{N}(0,1)$ distribution for the unemployment variable:

However, we see that this distribution does not correspond to the one that would truly normalize the rest of the data.

This is an illustration of a classic problem in machine learning, called *data drift*, which occurs when we attempt to extrapolate to data whose distribution no longer corresponds to that of the training data. This situation typically arises when an algorithm has been trained on a biased sample of the population or when we have non-stationary time series data. It is, therefore, essential to carefully consider the construction of the training sample and the potential for extrapolation to a broader population: the external validity of the model—whether data preparation or learning algorithm—can be null if this step was rushed.

> ! Data Drift
>
> Data drift refers to a shift in data distribution over time, leading to a degradation in the performance of a machine learning model that, by design, was trained on past data.
>
> This phenomenon can occur due to changes in the target population, shifts in data characteristics, or external factors.

Detecting data drift is crucial to adjust or retrain the model, ensuring its relevance and accuracy. Detection techniques include statistical tests and monitoring specific metrics.

4.1.b Normalization

Normalization is the process of transforming data to achieve a unit norm (ℓ_1 or ℓ_2). In other words, with the appropriate norm, the sum of elements equals 1. By default, `Scikit` uses an ℓ_2 norm. This transformation is especially useful in text classification or clustering.

This is also an opportunity to explore how to split data into multiple samples using the `train_test_split` function in `Scikit`. We will create a 70% sample of the data to estimate normalization parameters (training phase) and extrapolate to the remaining 30%. This split is fairly standard but, of course, adaptable depending on the project. The advantage of using `train_test_split` instead of manually sampling with `Pandas`' `sample` method is that `Scikit`'s function allows for much more control over sampling, particularly if stratification is desired, while being reliable. Doing this manually can be tedious and risky, as it is potentially complex to implement without errors.

Exercise 4: Normalization

1. Using the documentation for the `train_test_split` function in `Scikit`, create two samples (70% and 30% of the data, respectively).
2. Normalize the `Median_Household_Income_2021` variable (do not overwrite the values!) and examine the histogram before and after normalization.
3. Verify that the ℓ_2 norm is indeed equal to 1 (using the `np.linalg.norm` function with the `axis=1` argument) for the first 10 observations in the training set and then for the other observations.

```
/__w/python-datascientist/python-datascientist/.venv/lib/python3.13/site-packages/plotnine/
stats/stat_bin.py:112: PlotnineWarning: 'stat_bin()' using 'bins = 51'. Pick better value with
'binwidth'.
```

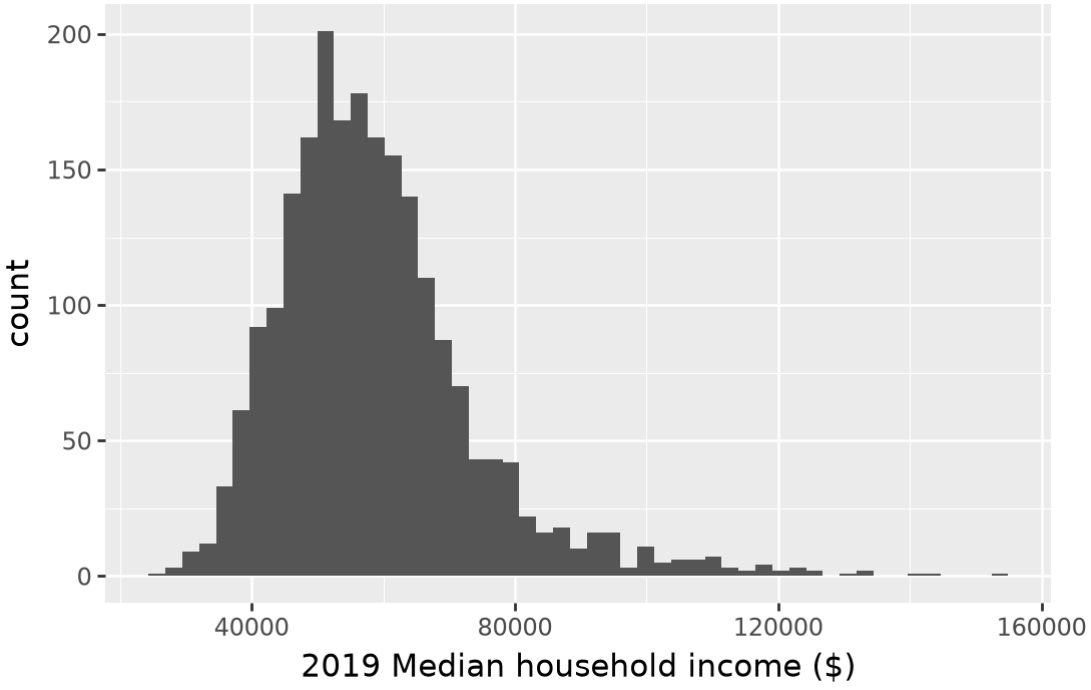


Figure 5: Question 2, before normalization

```
/__w/python-datascientist/python-datascientist/.venv/lib/python3.13/site-packages/plotnine/
stats/stat_bin.py:112: PlotnineWarning: 'stat_bin()' using 'bins = 114'. Pick better value with
'binwidth'.
```

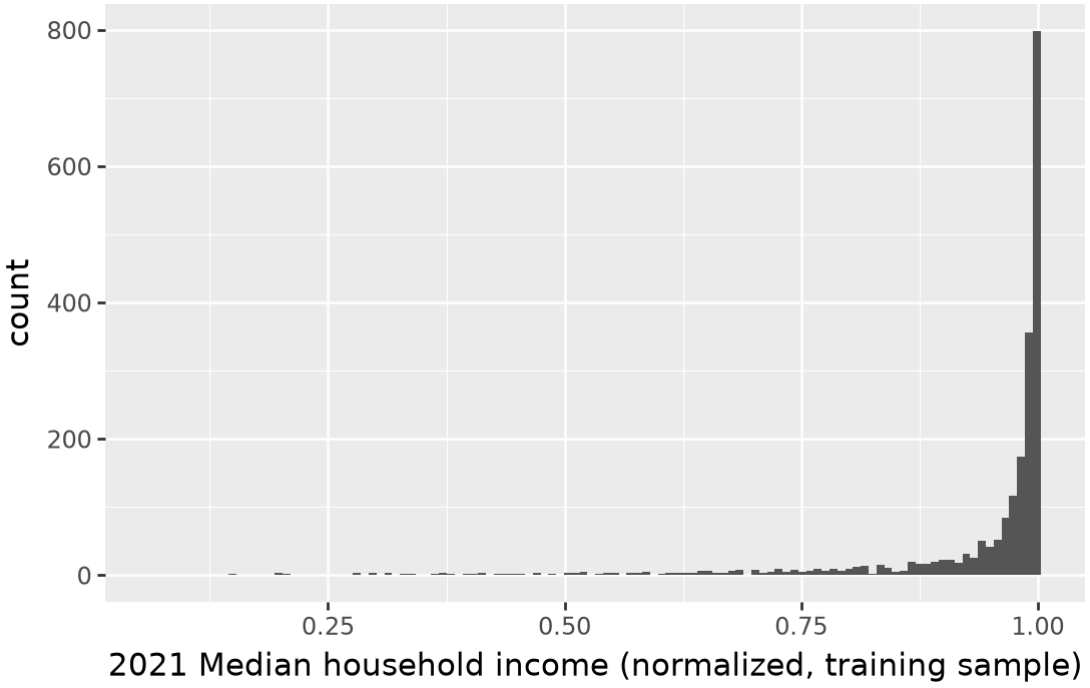


Figure 6: Question 2, transformed variable, on the normalization sample

```
/__w/python-datascientist/python-datascientist/.venv/lib/python3.13/site-packages/plotnine/
stats/stat_bin.py:112: PlotnineWarning: 'stat_bin()' using 'bins = 90'. Pick better value with
'binwidth'.
```

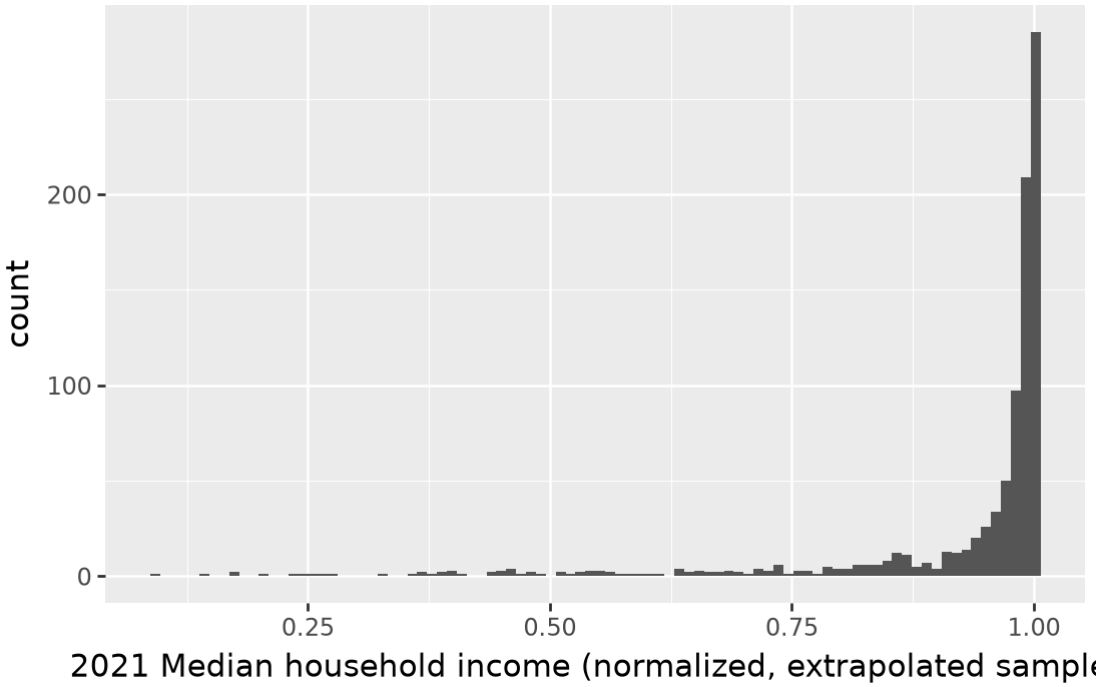


Figure 7: Question 2, transformed variable, using the trained parameters

Finally, if we compute the norm, we obtain the expected result on both the train sample and the extrapolated sample.

	X_train_norm2	X_test_norm2
0	1.0	1.0
1	1.0	1.0
2	1.0	1.0
3	1.0	1.0
4	1.0	1.0

4.2 Encoding Categorical Values

Categorical data must be recoded into numeric values to be integrated into machine learning models.

This can be done in several ways with `Scikit`:

- `LabelEncoder`: transforms a vector `["a","b","c"]` into a numeric vector `[0,1,2]`. This approach has the drawback of introducing an order to the categories, which is not always desirable.
- `OrdinalEncoder`: a generalized version of `LabelEncoder` designed to apply to matrices (X), while `LabelEncoder` applies mainly to a vector (y).

For one-hot encoding, several methods are available:

- `pandas.get_dummies` performs a dummy expansion. A vector of size n with K categories will be transformed into a matrix of size $n \times K$, where each column represents a dummy variable for category k. There are K categories, resulting in multicollinearity. In linear regression with a constant, one category should be removed before estimation.
- `OneHotEncoder` is a generalized (and optimized) version of dummy expansion. This is the recommended method.

4.3 Imputation

Data often contains missing values, that is, observations in our *DataFrame* containing a `NaN`. These gaps can cause bugs or misinterpretations when moving to modeling. One initial approach could be to remove all observations with a `NaN` in at least one column. However, if our table contains many `NaN`s, or if these are spread across numerous columns, we risk removing a significant number of rows, and, with that, losing important information, as missing values are rarely randomly distributed.

While this solution remains viable in many cases, a more robust approach called *imputation* exists. This method involves replacing missing values with a specified value. For example:

- Mean imputation: replacing all `NaN`s in a column with the column's average;
- Median imputation on the same principle, or using the most frequent column value for categorical variables;
- Regression imputation: using other variables to interpolate an appropriate replacement value.

More complex methods are available, but in many cases, the above approaches can provide much more satisfactory results. The `Scikit` package makes imputation very straightforward (documentation here).

4.4 Handling Outliers

Outliers are observations that significantly deviate from the general trend of other observations in a dataset. In other words, they are data points that stand out unusually from the overall data

distribution. This may be due to data entry errors, respondents who incorrectly answered a survey, or simply extreme values that may bias a model too much.

For example, these could be 3 individuals measuring over 4 meters in height within a population or household incomes exceeding 10 million euros per month at a national level.

It is good practice to routinely examine the distribution of available variables to check if some values deviate too significantly from others. Sometimes these values will interest us, for instance, if we are focusing solely on very high incomes (top 0.1%) in France. However, often these values will be more of a hindrance, especially if they don't make sense in the real world.

If we find that the presence of these extreme values or *outliers* in our dataset is more problematic than helpful, it is reasonable to simply remove them. Most of the time, we set a percentage of data to remove, such as 0.1%, 1%, or 5%, then remove the corresponding extreme values in both tails of the distribution.

Several packages can perform these operations, which can become complex if we examine outliers across multiple variables. The `IsolationForest()` function in the `sklearn.ensemble` package is particularly noteworthy.

4.5 Application exercise

🔥 Be careful with new categories!

Transformers create a mapping between text categories and numeric values. This assumes that the data used to build this mapping includes all possible values for the text categories.

However, if new categories appear, the classifier will not know how to transform these into numeric values, which will cause an error in `Scikit`. This technical error makes sense, as it would require updating not only the mapping but also the estimation of underlying parameters.

💡 Exercise 5: Encoding Categorical Variables

1. Create `df` that retains only the `state_name` and `county_name` variables in `votes`.
2. Apply a `LabelEncoder` to `state_name` *Note: The label encoding result is relatively intuitive, especially when related to the initial vector.*
3. Observe the dummy expansion of `state_name`
4. Apply an `OrdinalEncoder` to `df.loc[:, ['state_name', 'county_name']]` *Note: The ordinal encoding result is consistent with the label encoding*
5. Apply a `OneHotEncoder` to `df.loc[:, ['state_name', 'county_name']]`

Note:`scikit` optimizes the object required to store the result of a transformation model. For example, the One Hot encoding result is a very large object. In this case, `scikit` uses a Sparse matrix.

If we examine the labels and their numeric transpositions via `LabelEncoder`

```
array([[23, 'Missouri'],
       [25, 'Nebraska'],
       [30, 'New York'],
       ...,
       [41, 'Texas'],
       [41, 'Texas'],
       [41, 'Texas']], shape=(3108, 2), dtype=object)
```

	Alabama	Arizona	Arkansas	California	Colorado	Connecticut	Delaware	District of Columbia	Florida	Georgia	...	South Dakota	Tennessee	Texas	Utah	Vermont	Virginia	Washington	West Virginia	Wisconsin	Wyoming
0	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	...	False	True	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
3103	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
3104	False	False	False	False	False	False	False	False	False	False	...	False	False	False	False	False	False	False	False	False	False
3105	False	False	False	False	False	False	False	False	False	False	...	False	False	True	False	False	False	False	False	False	False
3106	False	False	False	False	False	False	False	False	False	False	...	False	False	True	False	False	False	False	False	False	False
3107	False	False	False	False	False	False	False	False	False	False	...	False	False	True	False	False	False	False	False	False	False

If we examine the `OrdinalEncoder`:

```
array([23., 25., 30., ..., 41., 41., 41.], shape=(3108,))
```

```
<Compressed Sparse Row sparse matrix of dtype 'float64'
    with 6216 stored elements and shape (3108, 1892)>
```

Reference

Informations additionnelles

i Python environment

This site was built automatically through a `Github` action using the `Quarto` reproducible publishing software (version 1.10.18).

The environment used to obtain the results is reproducible via uv. The `pyproject.toml` file used to build this environment is available on the `linogaliana/python-datascientist` repository

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
    "folium≥0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables≥0.12.0",
    "gt-extras≥0.0.8",
    "ipykernel≥6.29.5",
    "jupyter≥1.1.1",
    "jupyter-cache≥1.0.0",
    "kaleido≥0.2.1",
    "langchain-community≥0.3.27",
    "loguru==0.7.3",
```

```
    "markdown>=3.8",
    "nbclient>=0.10.0",
    "nbformat>=5.10.4",
    "nltk>=3.9.1",
    "openpyxl>=3.1.5",
    "pandas>=3.0",
    "pip>=25.1.1",
    "plotly>=6.1.2",
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

To use exactly the same environment (version of `Python` and *packages*), please refer to the documentation for `uv`.

Bibliography

[1] Insee, “Guide de sémiologie cartographique,” 2018.